ARITICLE

How to Build OpenSSL, zlib, and cURL libraries on Windows

JUNE 02,2022

Author:



Jirapongse Phuriphanvichai
Developer Advocate

Updated: 04 Feb 2025

OpenSSL, zlib, and cURL are open-source computer software projects to secure communications over computer networks, compress data, and transfer data through various network protocols, respectively. The projects are comprised of command-line tools, header files, and libraries. They are widely used by many systems, applications, and libraries including LSEG Real-Time SDK C/C++. Users can utilize available OpenSSL, zlib, and cURL libraries available on the Internet or build the latest versions of those libraries from the source code.

LSEG Real-Time SDK C/C++ is a suite of modern and open-source APIs that is designed to simplify development through a strong focus on ease of use and standardized access to a broad set of LSEG and proprietary content and services. It uses these libraries to connect to encrypted servers or Real-Time Optimized on the cloud.

This article demonstrates step by step to build OpenSSL, zlib, and cURL libraries from the source code on a Windows machine (Windows 10). Then, it shows how to use these libraries in LSEG Real-Time SDK C/C++. The topics include:

- Build OpenSSL Libraries
- Build zlib libraries
- Build cURL Libraries
- Use the Libraries with LSEG Real-Time SDK C/C++

Prerequisites

To follow the steps in this article, you need to have a Windows 10 machine with the following applications installed.

1. **Microsoft Visual Studio:** Desktop Development with C++ (such as, Visual Studio 2019 or 2022)
2. **Strawberry Perl:** A perl environment for Microsoft Windows (Required to build OpenSSL)
3. **Netwide Assembler (NASM):** An assembler for the x86 CPU architecture (Required to build OpenSSL)
4. A decompression tool, such as WinRAR

Build OpenSSL Libraries

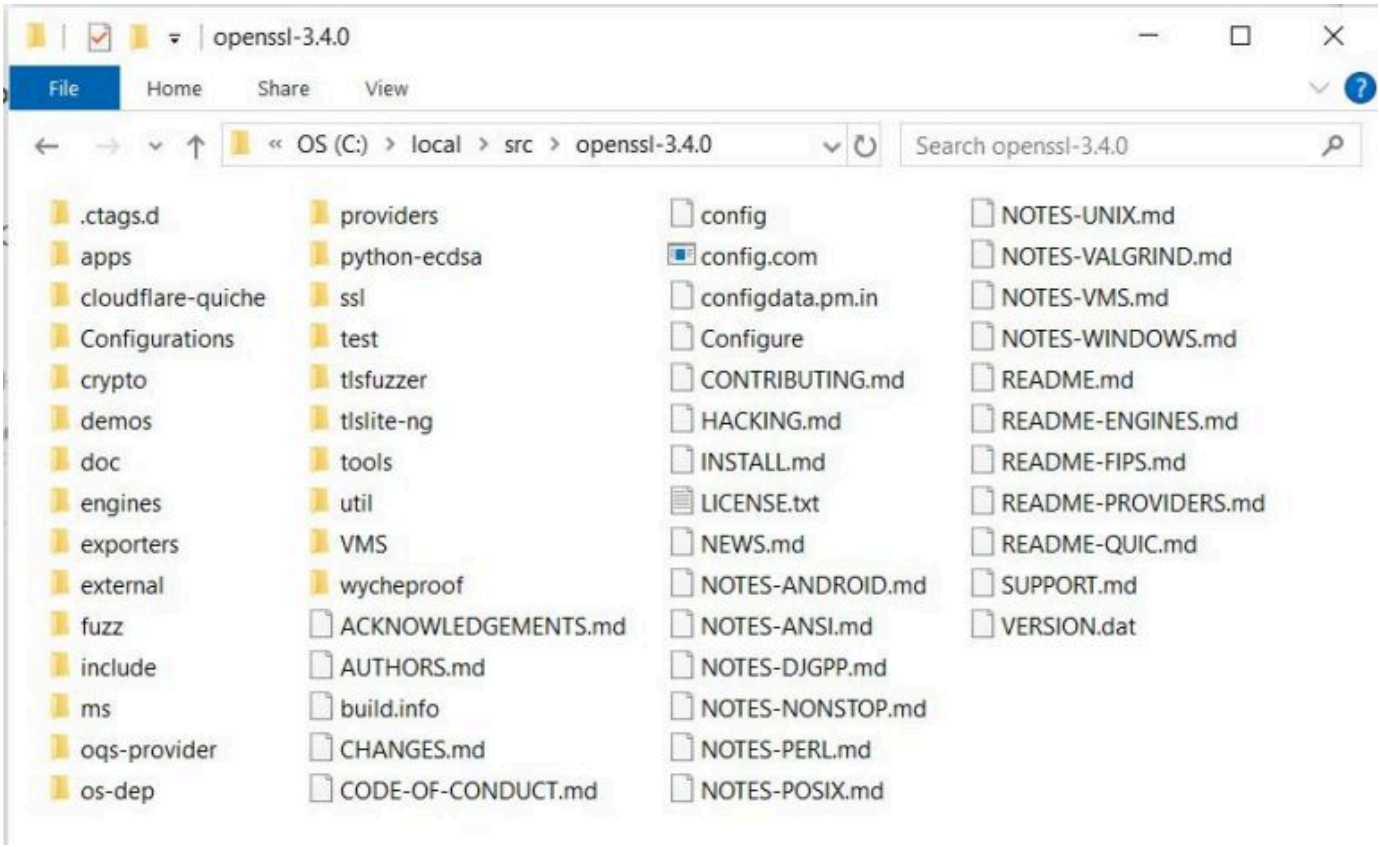
OpenSSL is a software library implementing the SSL and TLS protocols for applications that secure communications over computer networks. It is widely used by Internet servers, including the majority of HTTPS websites. It contains two libraries which are libssl and libcrypto. The libcrypto library provides the fundamental cryptographic routines used by libssl.

The following steps show how to build the OpenSSL libraries and a binary file from the OpenSSL source package on a Windows 10 machine.

1. Download and decompress the OpenSSL source code package

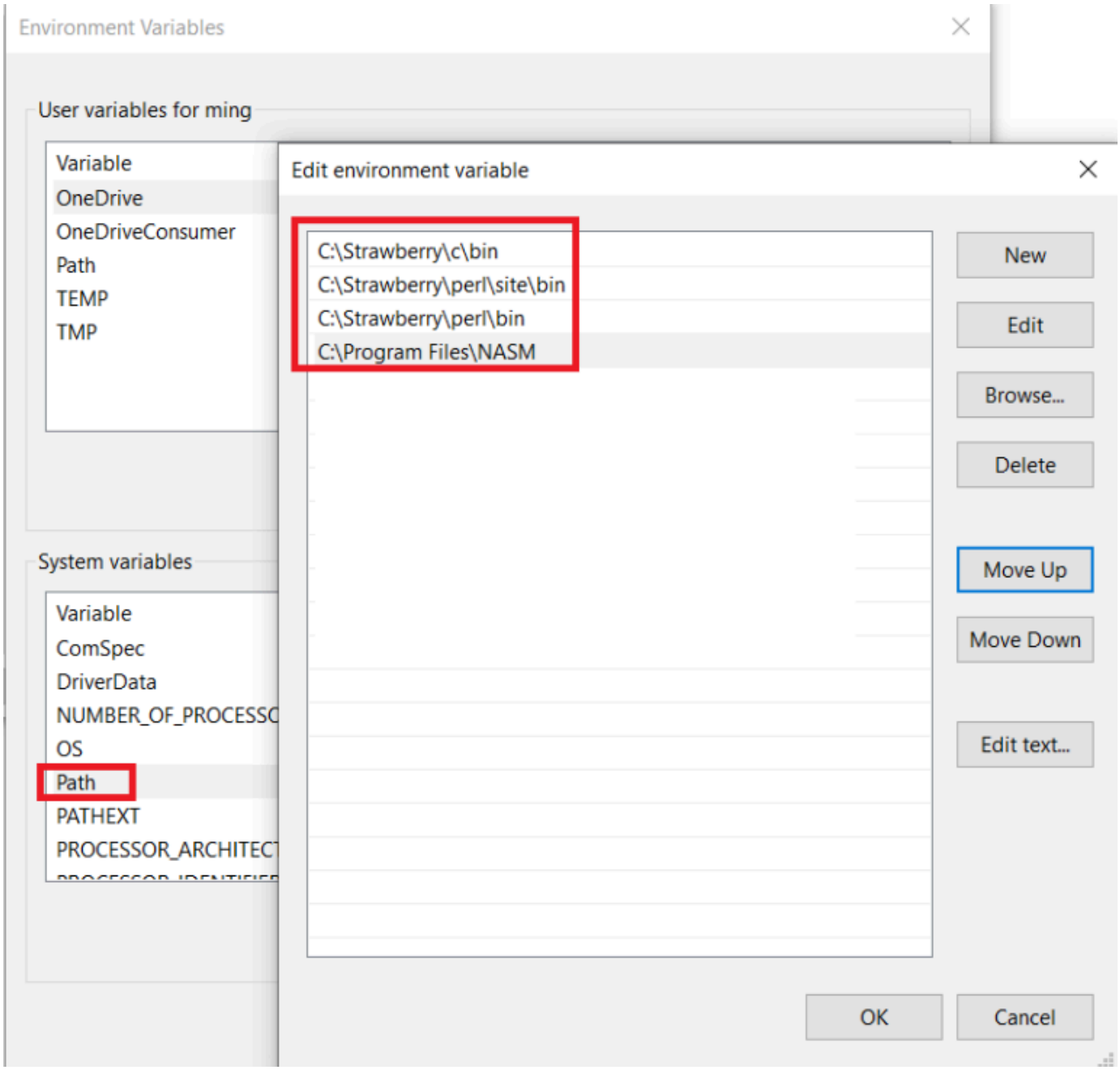
The OpenSSL is open source and the source code is available on the OpenSSL official website (https://www.openssl.org). At the time of this writing, the current versions of the OpenSSL package is **openssl-.3.4.0**.

First, download the OpenSSL source package (**openssl-3.4.0.tar.gz**) and decompress it to the local directory, such as **c:\local\src**.



2. Configure the package

To follow this step, the Perl and NASM directory must be added to the **Path** environment variable.



You can test it my running "**perl -v**" and "**nasm -v**" on the command prompt.

```
perl -v
...
nasm -v
```

Copy

```
x64 Native Tools Command Prompt for VS 2022

c:\>perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread

Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.


c:\>nasm -v
NASM version 2.15.05 compiled on Aug 28 2020

c:\>_
```

From the previous step, all files are extracted in the **openssl-3.4.0** directory. Run the **x64 Native Tools Command Prompt for Visual Studio** and then change the directory to **C:\local\src\openssl-3.4.0**.

```
x64 Native Tools Command Prompt for VS 2022

C:\local\src\openssl-3.4.0>dir /w
 Volume in drive C is OS
 Volume Serial Number is 4A16-1A34

 Directory of C:\local\src\openssl-3.4.0

[.]                 [..]                [.ctags.d]          ACKNOWLEDGEMENTS.md
[apps]              AUTHORS.md          build.info          CHANGES.md
[cloudflare-quiche] CODE-OF-CONDUCT.md  config              config.com
configdata.pm.in    [Configurations]    Configure           CONTRIBUTING.md
[crypto]            [demos]             [doc]               [engines]
[exporters]         [external]          [fuzz]              HACKING.md
[include]           INSTALL.md          LICENSE.txt         [ms]
NEWS.md             NOTES-ANDROID.md    NOTES-ANSI.md       NOTES-DJGPP.md
NOTES-NONSTOP.md    NOTES-PERL.md       NOTES-POSIX.md      NOTES-UNIX.md
NOTES-VALGRIND.md   NOTES-VMS.md        NOTES-WINDOWS.md    [oqs-provider]
[os-dep]            [providers]         [python-ecdsa]      README-ENGINES.md
README-FIPS.md      README-PROVIDERS.md README-QUIC.md      README.md
[ssl]               SUPPORT.md          [test]              [tlsfuzzer]
[tlslite-ng]        [tools]             [util]              VERSION.dat
[VMS]               [wycheproof]
              31 File(s)      1,225,902 bytes
              27 Dir(s)  62,156,648,448 bytes free

C:\local\src\openssl-3.4.0>
```

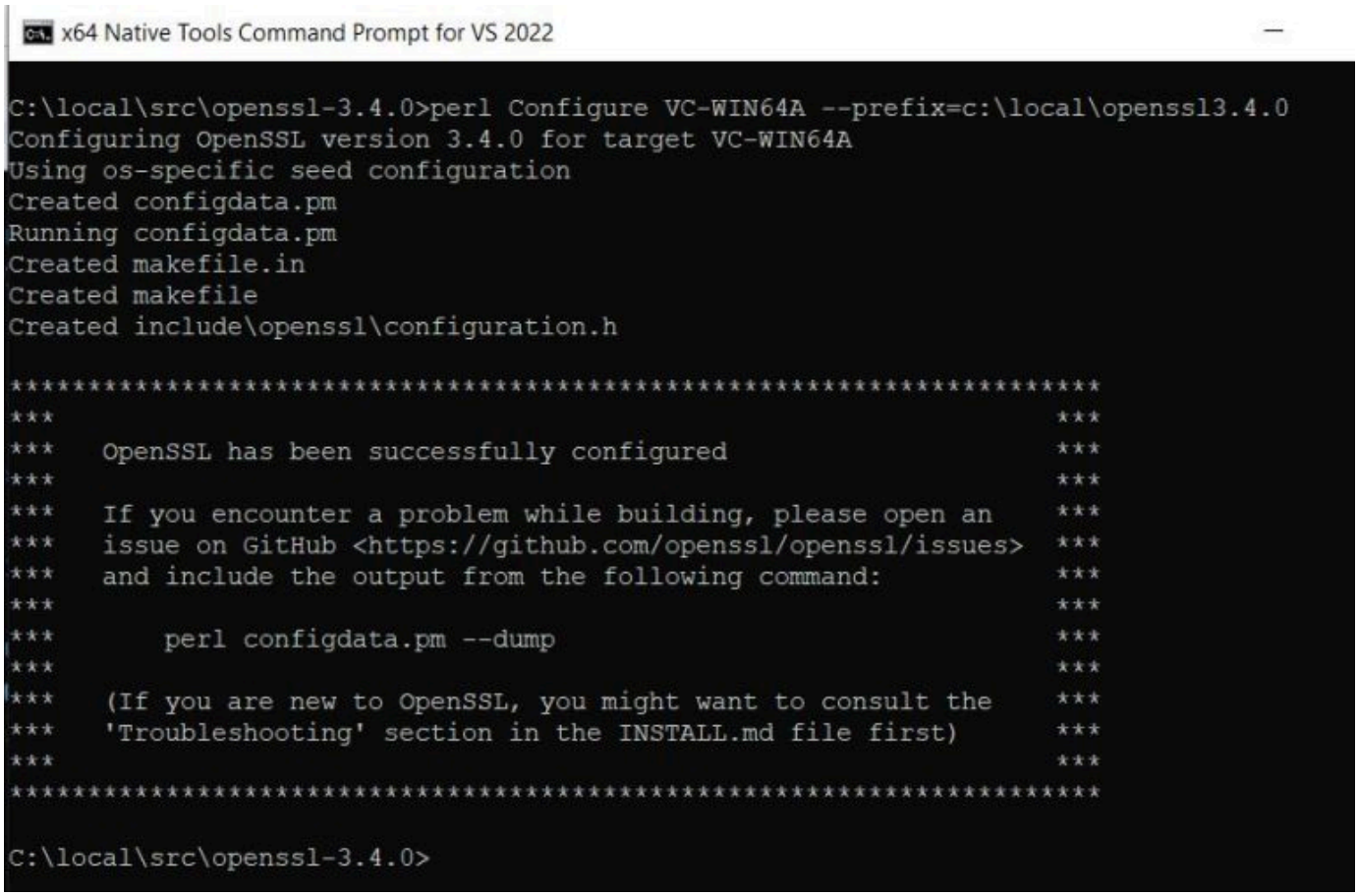
Then, run the **config** command to configure the OpenSSL source package.

```
perl Configure VC-WIN64A --prefix=c:\local\openssl3.4.0
```

Copy

I ran the **Configure** command with the following options.

Option	Description
--prefix= c:\local\openssl3.4.0	The top of the installation directory tree. The OpenSSL libraries will be created in this directory (c:\local\openssl3.4.0)



For other options, please refer to the **INSTALL.md** file in the OpenSSL source package.

```
Build and Install
=================

This document describes installation on all supported operating
systems: the Unix/Linux family (including macOS), OpenVMS,
and Windows.

Table of Contents
=================

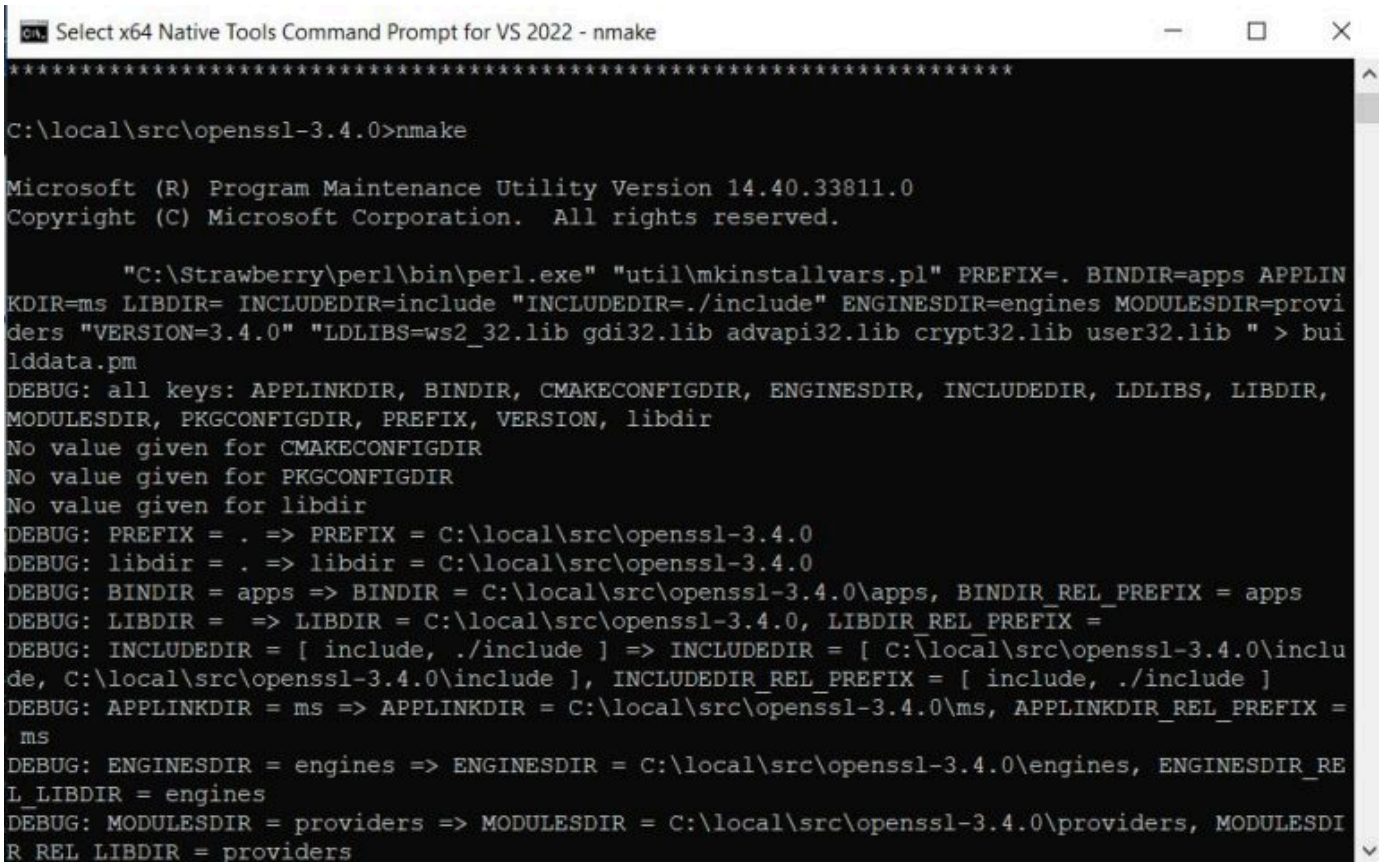
 - [Prerequisites](#prerequisites)
 - [Notational Conventions](#notational-conventions)
 - [Quick Installation Guide](#quick-installation-guide)
   - [Building OpenSSL](#building-openssl)
   - [Installing OpenSSL](#installing-openssl)
 - [Configuration Options](#configuration-options)
   - [API Level](#api-level)
   - [Cross Compile Prefix](#cross-compile-prefix)
   - [Build Type](#build-type)
   - [Directories](#directories)
   - [Compiler Warnings](#compiler-warnings)
   - [Compression Algorithm Flags](#compression-algorithm-flags)
   - [Seeding the Random Generator](#seeding-the-random-generator)
   - [Setting the FIPS HMAC key](#setting-the-FIPS-HMAC-key)
   - [Enable and Disable Features](#enable-and-disable-features)
   - [Displaying configuration data](#displaying-configuration-data)
 - [Installation Steps in Detail](#installation-steps-in-detail)
```

3. Build the OpenSSL libraries

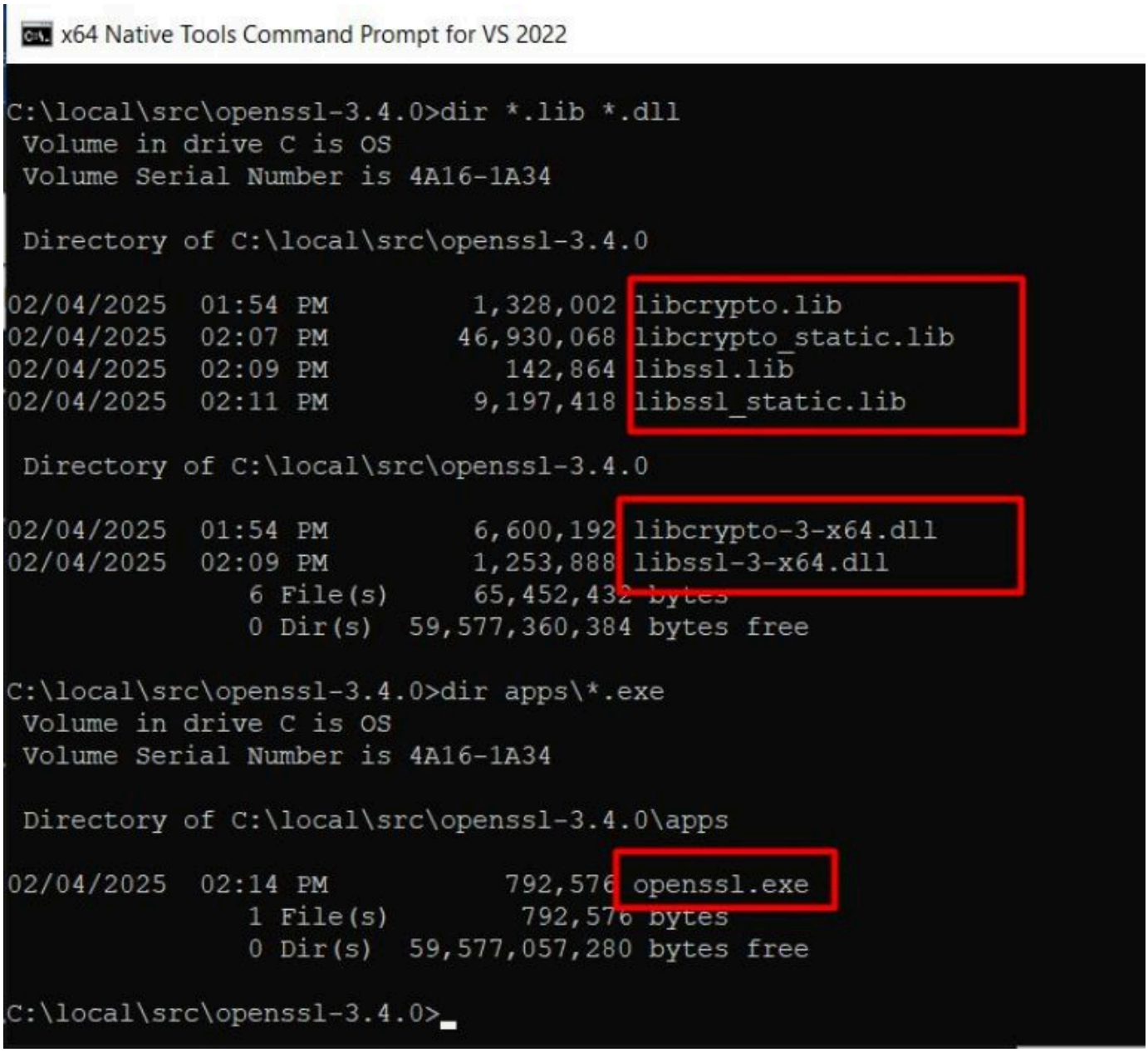
Run the **nmake** command to build the OpenSSL libraries (libcrypto and libssl) and the OpenSSL binary (openssl).

```
nmake
```

Copy



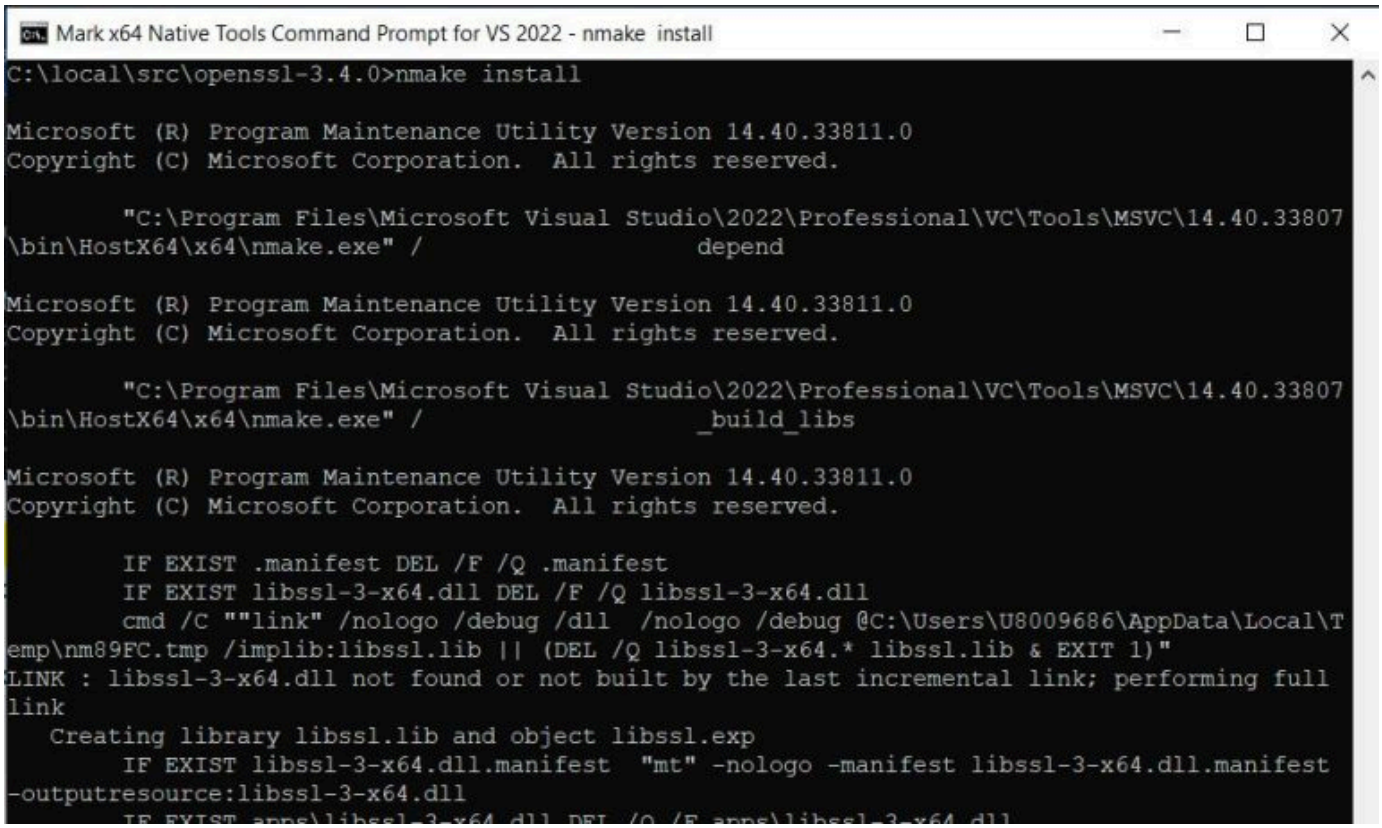
The libraries will be built in the top-level directory, and the binary will be in the **apps** subdirectory.



Finally, run the "**nmake install**" command to install OpenSSL on the machine.

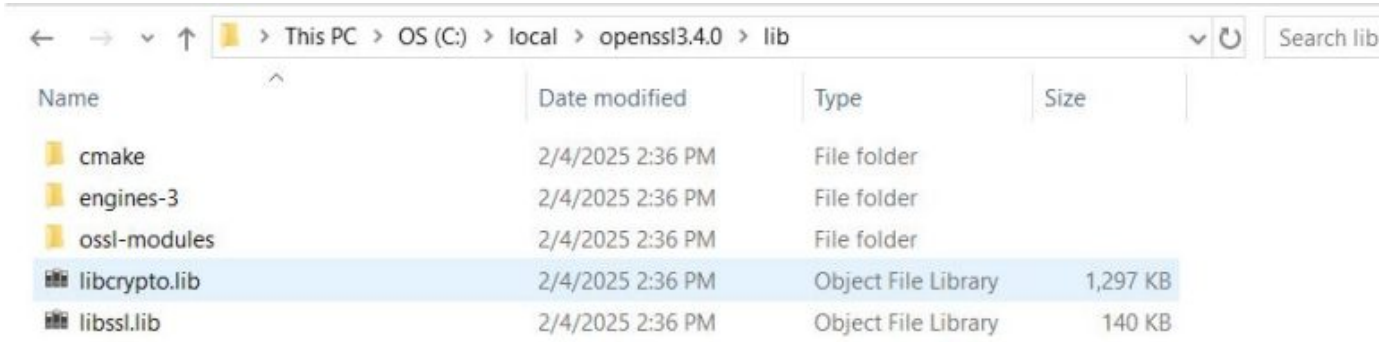
```
nmake install
```

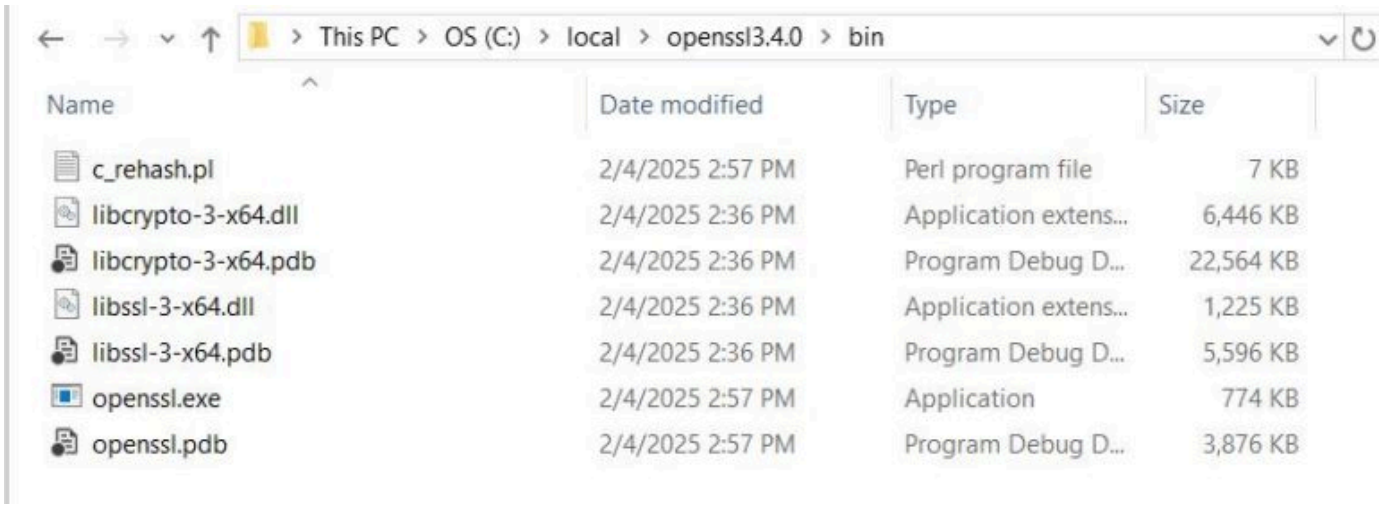
Copy



The command will install all the OpenSSL components in the directory specified in the **--prefix** option (**C:\local\openssl3.4.0**). OpenSSL configuration files are in the **C:\Program Files\Common Files\SSL** directory.

The libraries are in the **C:\local\openssl3.4.0\lib** and **C:\local\openssl3.4.0\bin** directory and the OpenSSL binary file is in the **C:\local\openssl3.4.0\bin** directory.





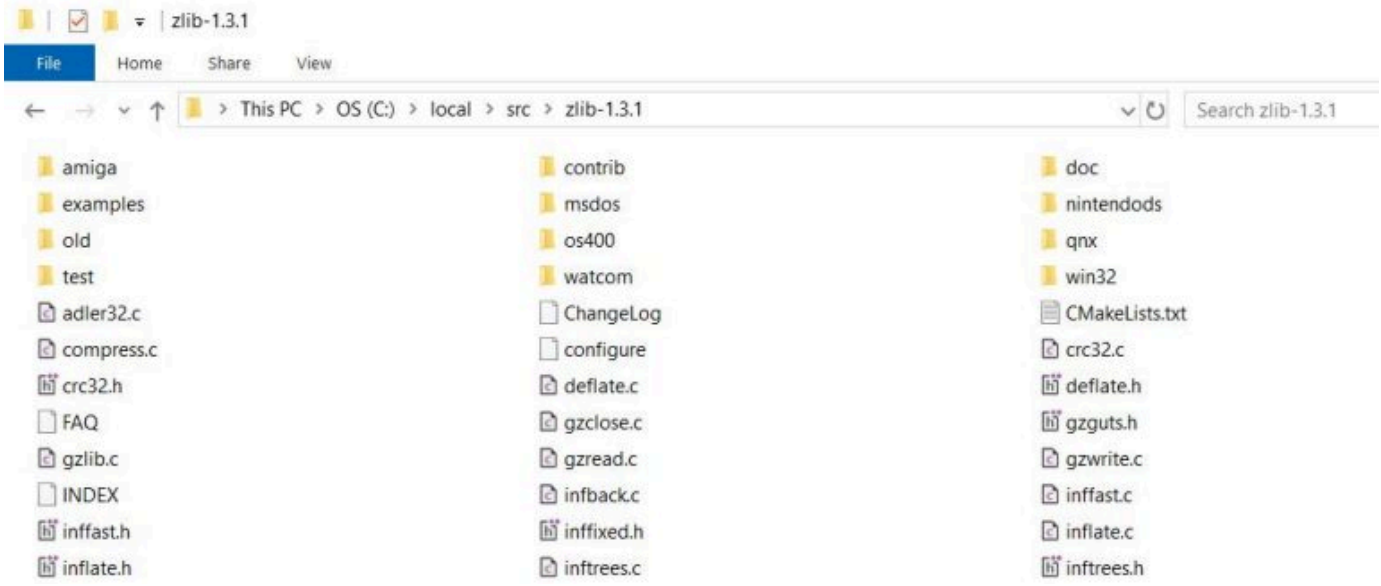
At this point, the OpenSSL binary and library files are built properly.

Build zlib Libraries

Zlib is a free and general-purpose lossless data-compression software library for use on any computer hardware and software platform, including Linux, macOS, and Windows. The zlib data format is itself portable across platforms. The following steps demonstrate how to build zlib libraries from the zlib source package on a Windows 10 machine.

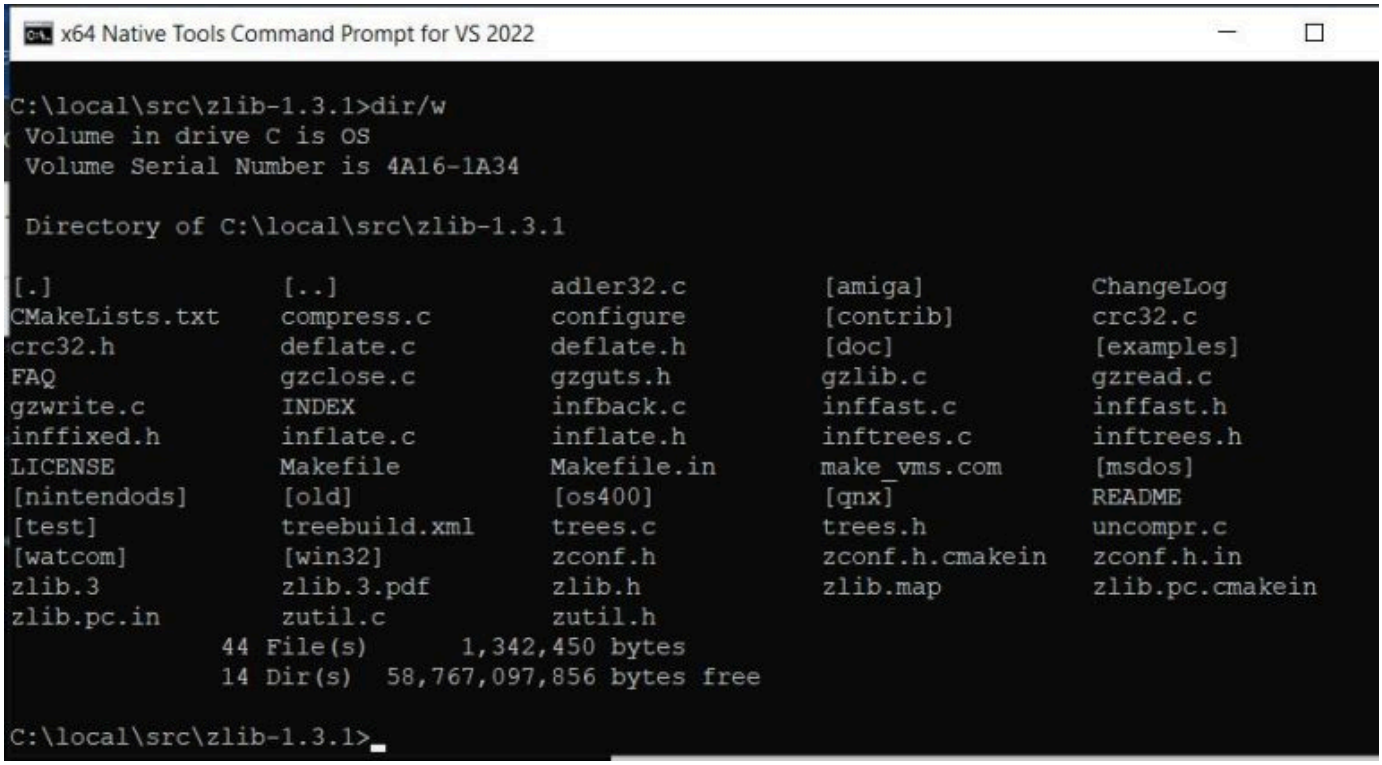
1. Download and decompress the zlib source code package

Zlib is open source and the source code is available on the zlib official website (https://zlib.net/). At the time of this writing, the current version of the zlib package is **zlib-1.3.1**. Download the zlib source package (**zlib-1.3.1.tar.gz**) and decompress it to the local directory, such as **C:\local\src**.



2. Build the zlib libraries

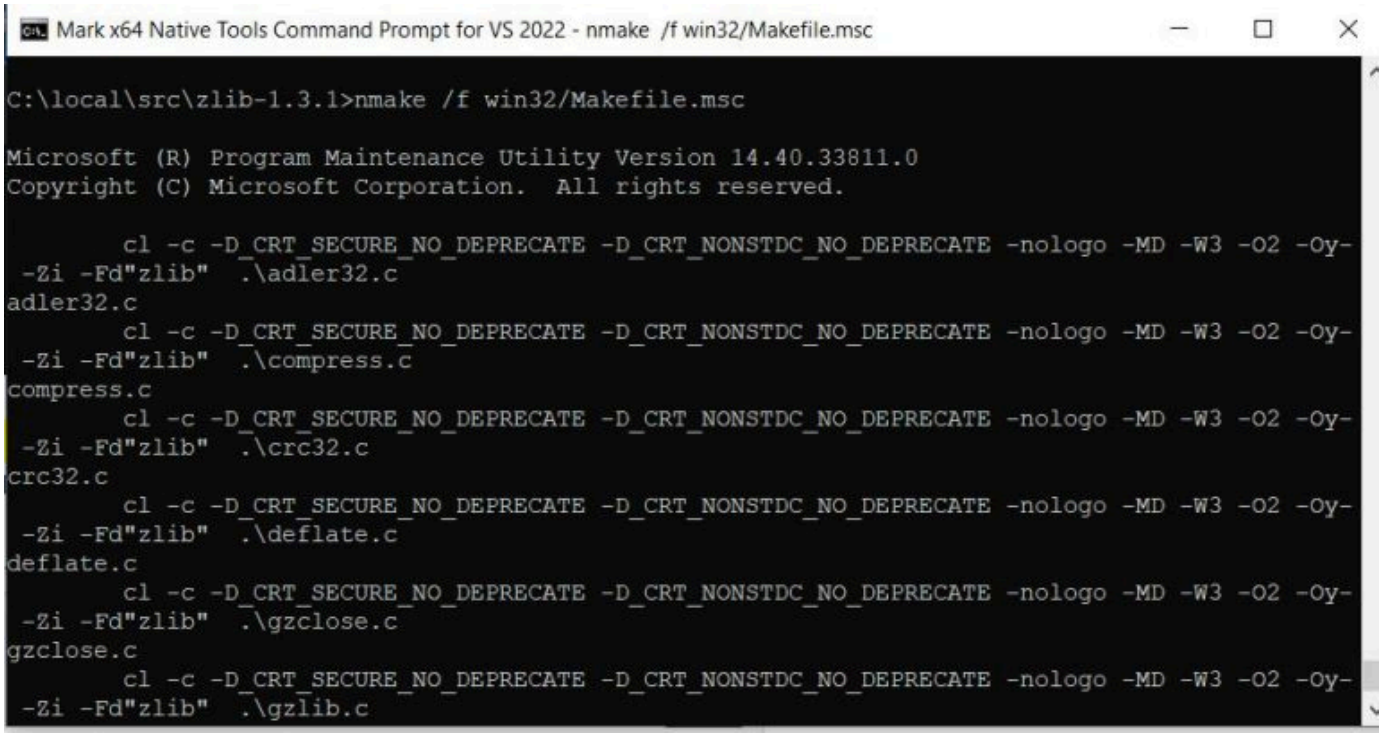
From the previous step, all files are extracted in the **zlib-1.3.1** directory. Run the **x64 Native Tools Command Prompt for Visual Studio** and then change the directory to **C:\loca\src\zlib-1.3.1**.



Then, run the **nmake** command to build the zlib libraries.

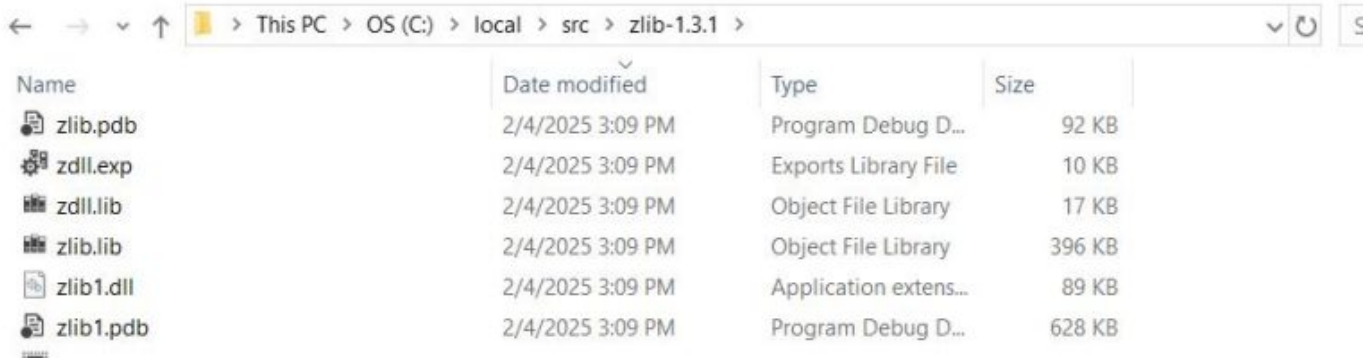
```
nmake /f win32/Makefile.msc
```

Copy



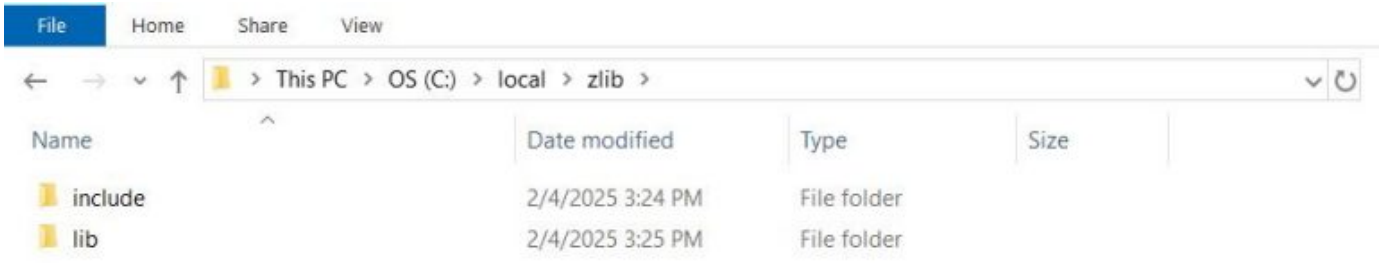
The following files will be created in the **C:\local\src\zlib-1.3.1** directory.

- **zlib.lib:** A zlib static library
- **zlib.pdb:** A program database file of zlib.lib
- **zdll.lib:** An import library of zlib1.dll
- **zlib1.dll:** A zlib shared library
- **zlib1.pdb:** A program database of zlib1.dll

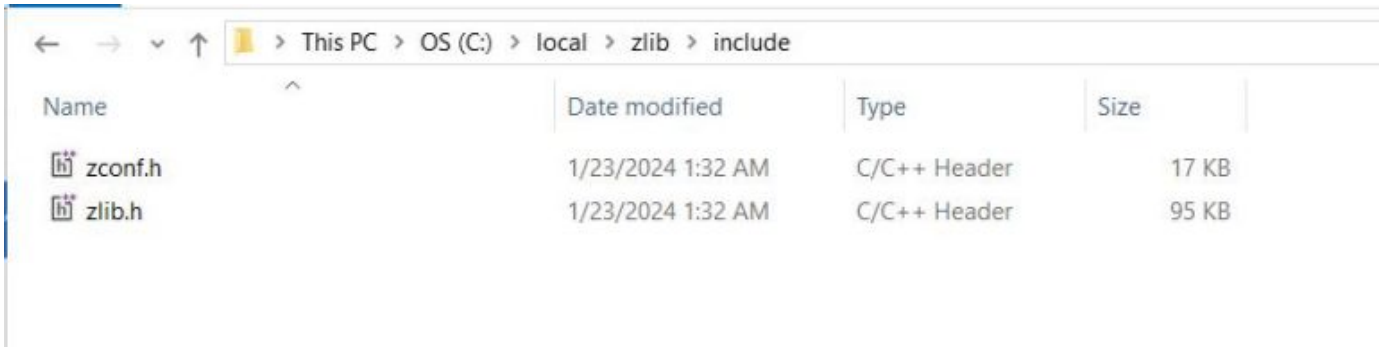


3. Copy zlib files

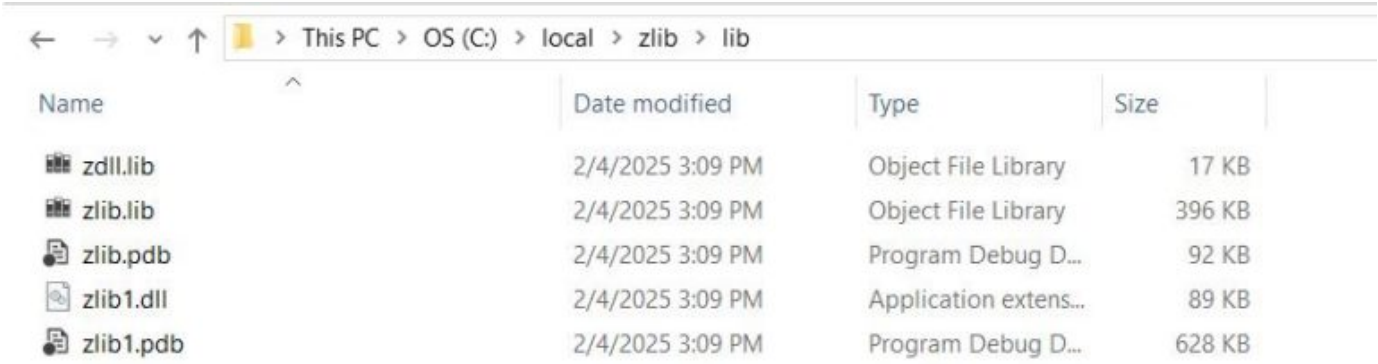
In this step, the zlib header files and library files will be copied to the new zlib folder. First, create a new folder for zlib files, such as **C:\local\zlib**.



Then, **copy zlib.h** and **zconf.h** files from **C:\local\src\zlib-1.3.1** to **C:\local\zlib\include**.



Finally, copy the zlib library files from **C:\local\src\zlib-1.3.1** to **C:\local\zlib\lib** directory.



At this point, the zlib library files are built properly.

Build cURL Library

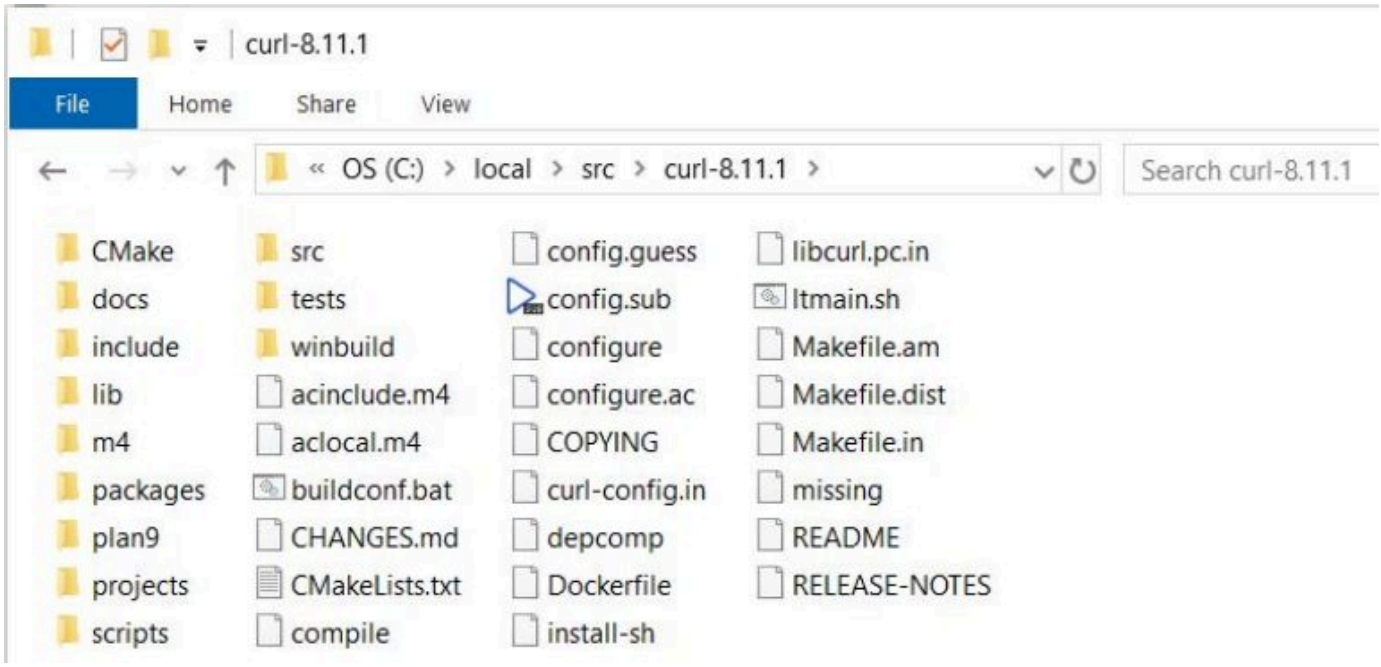
cURL is a computer software project providing a library and a command-line tool for transferring data using various network protocols. It builds and works identically on many platforms. cURL supports HTTPS and performs SSL certificate verification by default when a secure protocol is specified such as HTTPS. It relies on the OpenSSL libraries to perform SSL certificate verification.

The following steps show how to build a cURL library and cURL binary file from the cURL source package on a Windows 10 machine. Moreover, the cURL library and cURL binary file will use the zlib and OpenSSL libraries created in the previous sections.

1. Download and decompress the cURL source code package

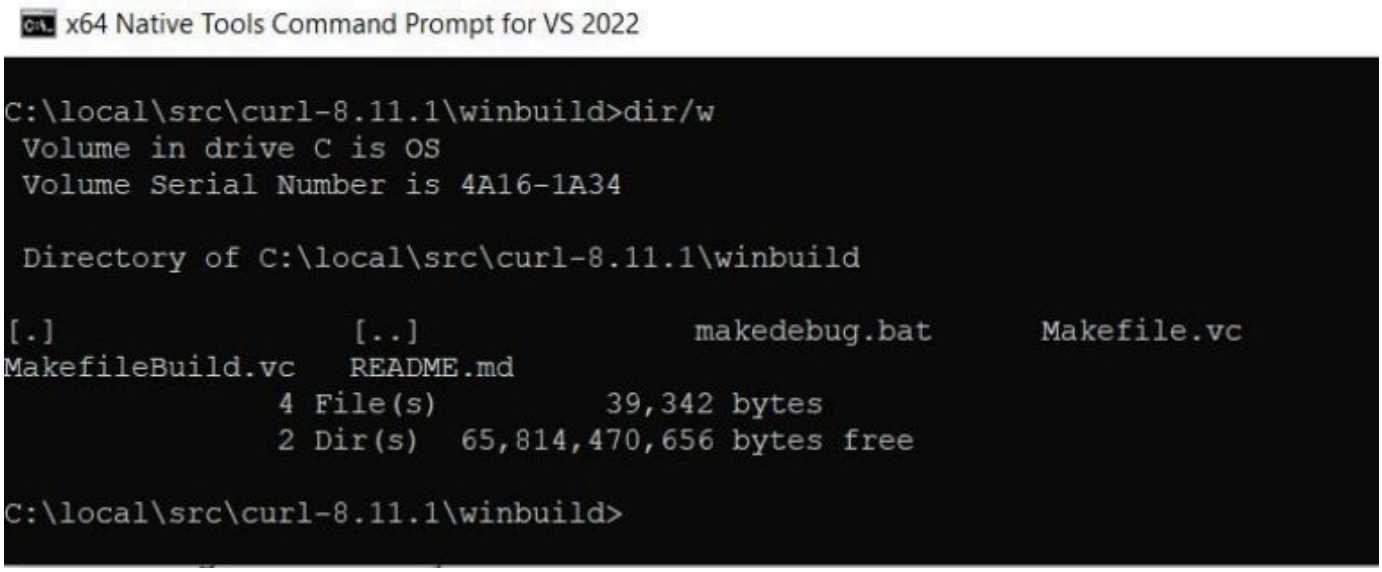
cURL is open source and the source code is available on the cURL official website (https://curl.se/download.html). At the time of this writing, the current version of the cURL package is **curl-8.11.1**.

First, download the cURL source package (**curl-8.11.1.tar.gz**) and decompress it to the local directory, such as **C:\local\src**.



2. Build the cURL libraries

From the previous step, all files are extracted in the **curl-8.11.1** directory. Run the **x64 Native Tools Command Prompt for Visual Studio** and then change the directory to **C:\local\src\curl-8.11.1\winbuild**.



cURL supports many underlying protocols. We can configure it when building the libraries. The following section demonstrates the ways to build the cURL libraries that support Schannel and OpenSSL, respectively.

2.1 Build the cURL libraries that use Schannel and zlib

The Microsoft Secure Channel (Schannel) is a security package that facilitates the use of Secure Sockets Layer (SSL) and/or Transport Layer Security (TLS) encryption on Windows platforms. Schannel will use certificates from the Windows certificate store.

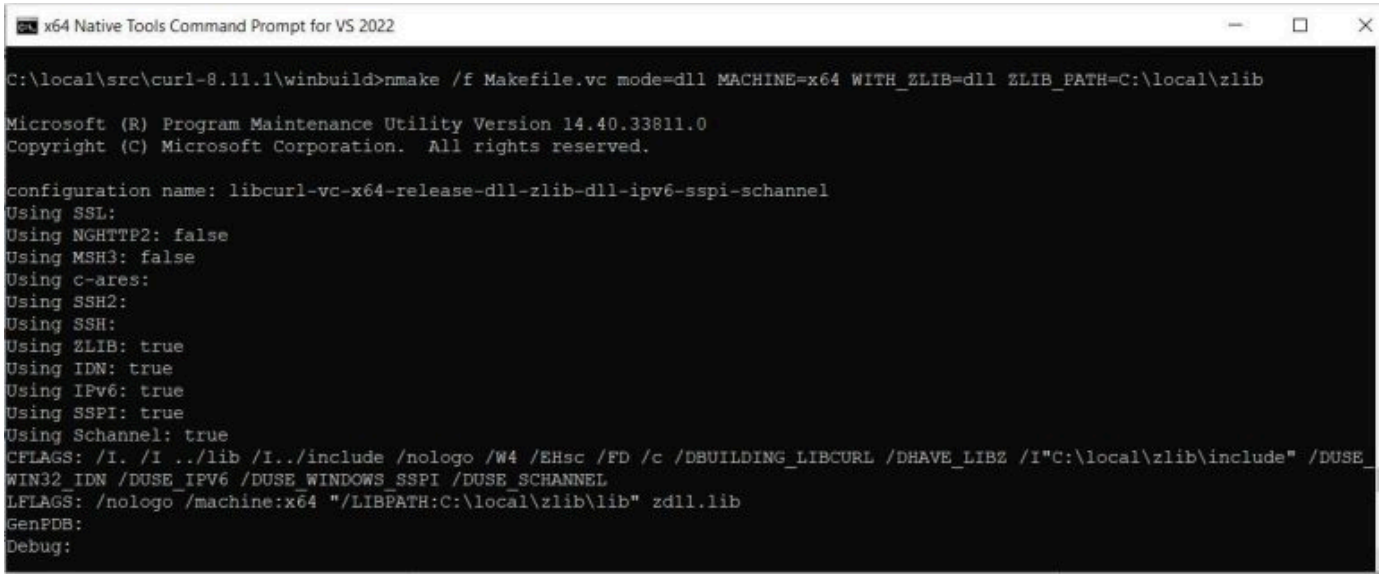
Run the following **nmake** command to build the cURL libraries and the cURL binary that use Schannel and zlib.

```
nmake /f Makefile.vc mode=dll MACHINE=x64 WITH_ZLIB=dll ZLIB_PATH=C:\local\zlib
```

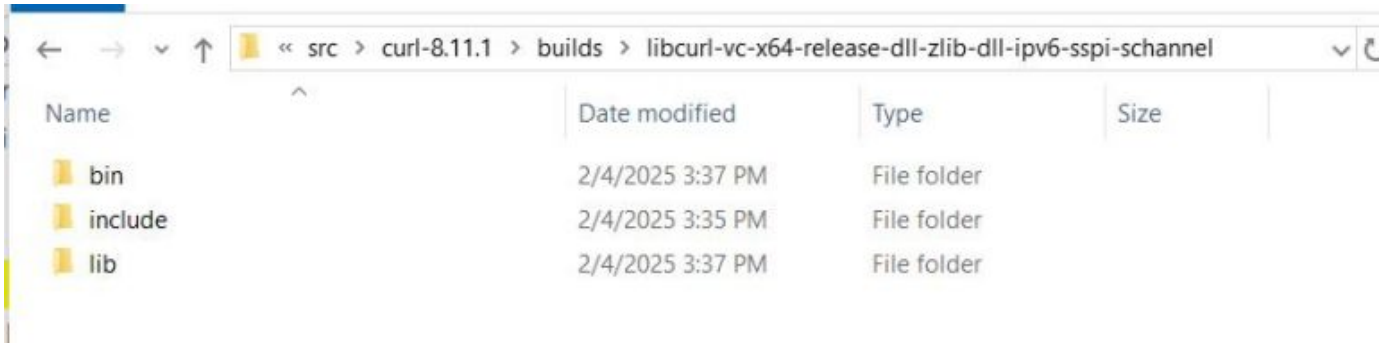
Copy

I ran the **nmake** command with the following options.

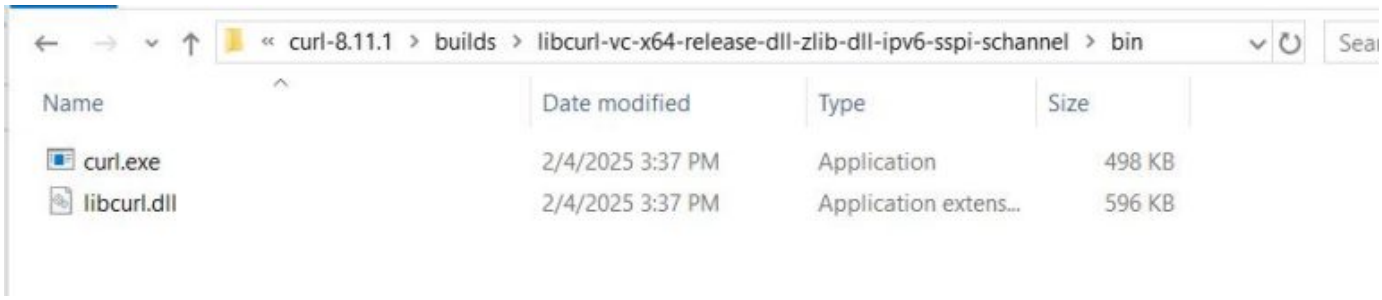
Option	Description
mode=dll	Build a cURL dynamic library
MACHINE=x64	Target architecture (default is x86)
WITH_ZLIB=dll	Enable zlib support, DLL or static
ZLIB_PATH=C:\local\zlib	Custom path for zlib (C:\local\zlib)



cURL files are built in the **.\builds\libcurl-vc-x64-release-dll-zlib-dll-ipv6-sspi-schannel** directory.



The cURL and DLL files are in the **bin** directory.



2.2 Build the cURL libraries that use OpenSSL and zlib

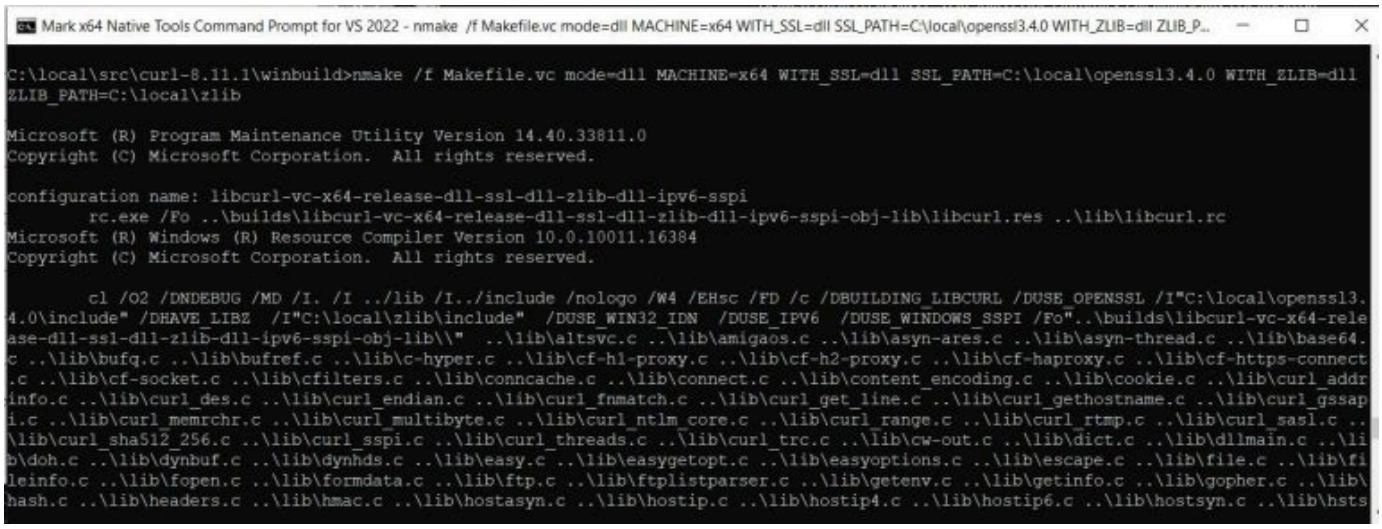
cURL can be built to support OpenSSL. Run the following **nmake** command to build the cURL libraries and the cURL binary that use OpenSSL and zlib.

```
nmake /f Makefile.vc mode=dll MACHINE=x64 WITH_SSL=dll SSL_PATH=C:\local\openssl3.4.0 WITH_ZL
IB=dll ZLIB_PATH=C:\local\zlib
```

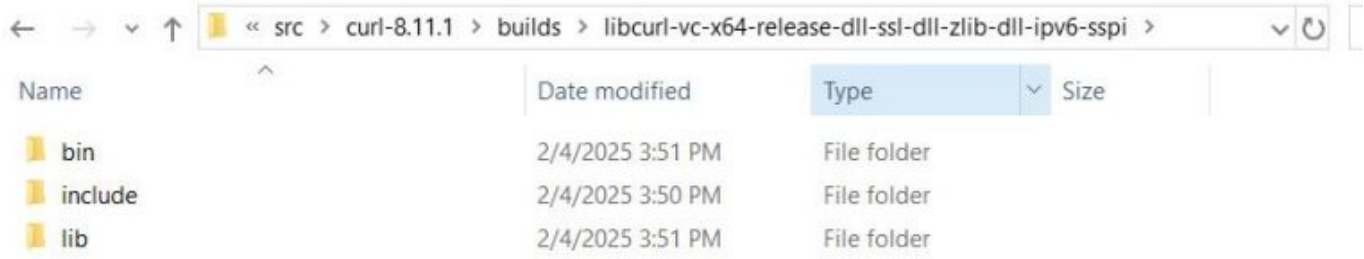
Copy

I ran the **nmake** command with the following options.

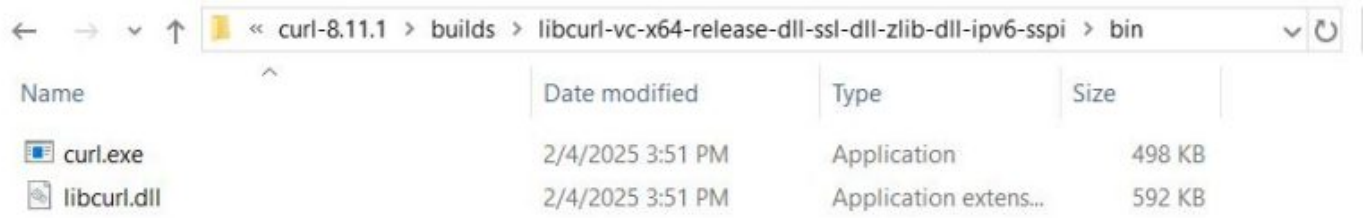
Option	Description
mode=dll	Build a cURL dynamic library
MACHINE=x64	Target architecture (default is x86)
WITH_SSL=dll	Enable OpenSSL support, DLL or static
SSL_PATH=C:\local\openssl3.4.0	Custom path for OpenSSL (C:\local\openssl3.4.0)
WITH_ZLIB=dll	Enable zlib support, DLL or static
ZLIB_PATH=C:\local\zlib	Custom path for zlib (C:\local\zlib)



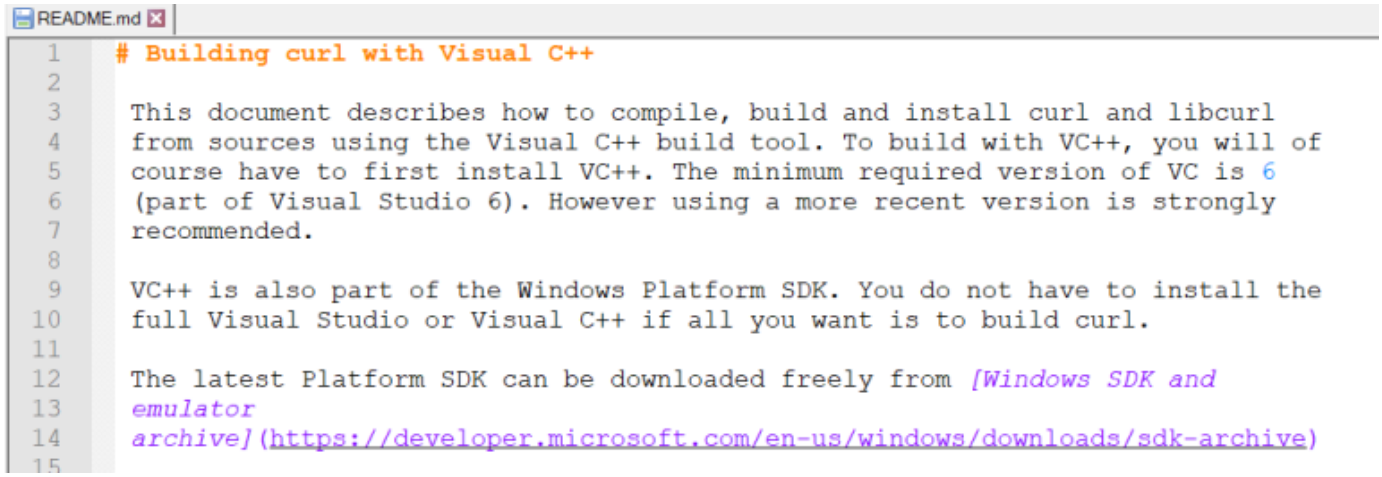
cURL files are built in the .**builds\libcurl-vc-x64-release-dll-ssl-dll-zlib-dll-ipv6-sspi** directory.



The cURL and DLL files are in the **bin** directory.



For other options, please refer to the **README.md** file in the **winbuild** directory.



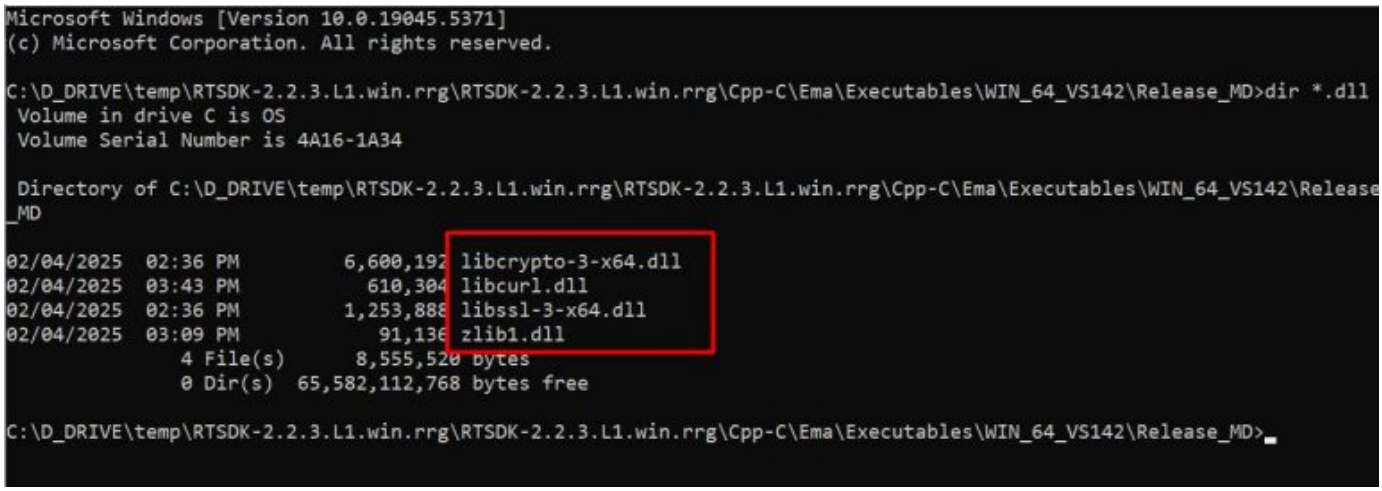
At this point, the cURL binary and library files are built properly.

Use the Libraries with Real-Time SDK C/C++

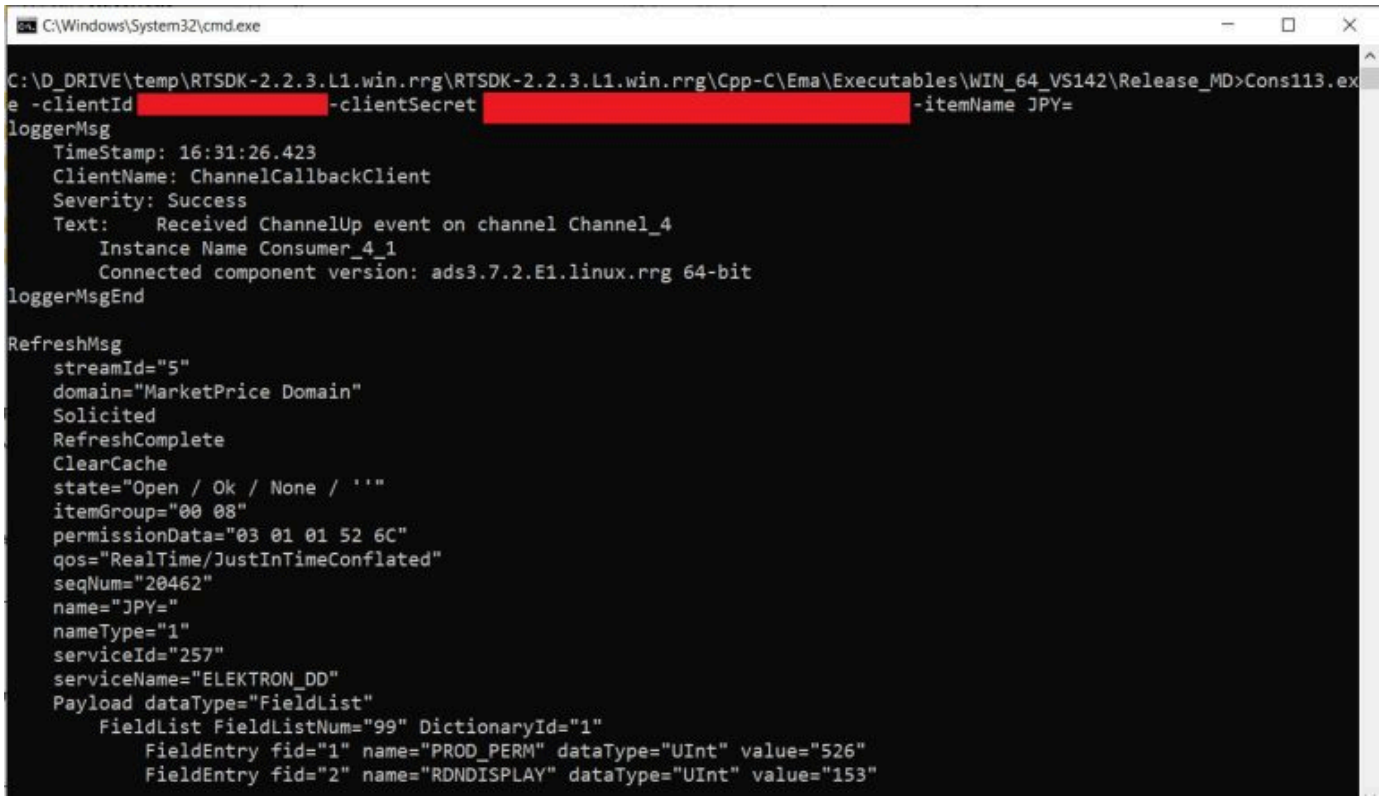
The OpenSSL, zlib, and cURL can be used with the Real-Time SDK C/C++ applications to connect to encrypted servers or Real-Time Optimized on the cloud. Real-Time SDK C/C++ on Windows uses libcurl that supports Schannel.

1. Copy the following files to the current directory of the Real-Time SDK C/C++ application.

- C:\local\zlib\lib\zlib1.dll
- C:\local\openssl3.4.0\bin\libcrypto-3-x64.dll
- C:\local\openssl3.4.0\bin\libssl-3-x64.dll
- C:\local\src\curl-8.11.1\builds\libcurl-vc-x64-release-dll-zlib-dll-ipv6-sspi-schannel\bin\libcurl.dll



2. Run the application. The application loads the OpenSSL, zlib, and cURL libraries found in the working directory and then connects to an encrypted server on Real-Time Optimized.



Summary

OpenSSL, zlib, and cURL are open-source and popular software projects used to secure communications over computer networks, compress data, and transfer data through various network protocols. Users can easily build the binary packages from the project's source code. With this method, users can get the latest version of the packages. This article demonstrates step by step to build OpenSSL, zlib, and curl libraries from the source code on a Windows 10 machine. There are a few steps to building the package. The first step is downloading the source code package from the official project website. The second step is decompressing the package. The third step is configuring the package and the final step is building the package. The outcome contains executable files, development libraries, and header files.

References

1. cURL. n.d. *cURL*. [online] Available at: <https://curl.se/> [Accessed 3 May 2022].

2. En.wikipedia.org. n.d. *cURL - Wikipedia*. [online] Available at: <https://en.wikipedia.org/wiki/CURL> [Accessed 3 May 2022].

3. OpenSSL. n.d. *OpenSSL*. [online] Available at: <https://www.openssl.org/> [Accessed 3 May 2022].

4. En.wikipedia.org. n.d. *OpenSSL - Wikipedia*. [online] Available at: <https://en.wikipedia.org/wiki/OpenSSL> [Accessed 3 May 2022].

5. Developers.lseg.com. n.d. *Real-Time C++ SDK | LSEG Developers*. [online] Available at: <https://developers.lseg.com/en/api-catalog/real-time-opnsrc/rt-sdk-cc> [Accessed 3 May 2022].

6. En.wikipedia.org. n.d. *zlib - Wikipedia*. [online] Available at: <https://en.wikipedia.org/wiki/Zlib> [Accessed 11 May 2022].

7. Zlib.net. n.d. *zlib Home Site*. [online] Available at: <https://zlib.net/> [Accessed 11 May 2022].

8. Bacon, M., n.d. *What is Microsoft Schannel (Microsoft Secure Channel)? - Definition from WhatIs.com*. [online] TechTarget. Available at: <https://www.techtarget.com/searchsecurity/definition/Microsoft-Schannel-Microsoft-Secure-Channel> [Accessed 24 May 2022].

RELATED ARTICLES

How to Build OpenSSL, zlib, and cURL Libraries on Linux
May 13, 2022 • Jirapongse Phuriphanvichai →

Introduction to Enterprise Message API
December 22, 2016 • Umer Nalla →

10 important things you need to know before you write a Real-Time application
January 04, 2018 • Olivier Davant →

Getting Started with Data Platform
November 06, 2020 • Veerapath Rungruengrayubkul →

How to Setup Amazon EC2 Machine Image for Real-Time - Optimized
May 01, 2019 • Wasin Waeosri →

Explore all Articles →

RELATED APIS

Real-time SDK C/C++



Explore all APIs →

Search

ABOUT US

MARKET INSIGHTS

CONTACT US

CONNECT WITH US

EXPLORE LSEG SITES

Terms of use

Privacy & Cookie Statement

Cookie 设置

Do not sell my info

Whistleblowing

UK Bribery Act

Modern Slavery Act